CTE711S Compiler Techniques

Week Five and Six:

Classes of Errors, Semantic Analysis and Abstract Syntax Tree (AST)

Prof A. A. Azeta (aazeta@nust.na)

March 2026

OUTLINE

1. The Phases of Compiler - So Far…
2. The Semantic Analysis
3. Abstract Syntax Tree (AST)
4. Decorating AST

```
/A C program segment to find the ratio of two numbers
main()
{
int b;
int a, b;
String c, d;
String s, ratio;
5: String k = “This is my first program;
Scanf(&a, &b);
s=(a+b)/2
Ration = a/b;
Prntf(ration);
Goto 5;
}
```

1. THE PHASES OF COMPILER - So Far …

1. Lexical Analysis (Scanning)
   - Detects inputs with illegal tokens
   - Incorrect spelling of keywords
     - e.g.: main $ ();

2. Syntax Analysis (Parsing)
   - Detects inputs with ill-formed parse trees
     - e.g.: missing semicolons ";"… missing brace " }"

3. Semantic Analysis
   - Last "front end" phase
   - Catches all remaining errors. e.g.: operand is incompatible to operator

The Classes of Errors

1. Lexical Errors
   - Detected by the Scanner
     - Illegal Character Input
     - Illegal Comments
     - Unterminated String constants
     - Infinite Loop

2. Syntax Errors
   - Detected by the Parser
     - Input is not a legal program, because it cannot be parsed by CFG
     - Example: a = * 5

The Classes of Errors (2)

3. Static Semantic Errors

- Detected by a Parser or in a Separate Semantic Analysis Passes
  - Input can be parsed by CFG but some non context-free error

    Examples:

    - Multiple declared variables
    - Undeclared variables
    - Function call with wrong number of arguments
    - Type mismatches

The Classes of Errors (3)

3. Static Semantic Errors:

- multiple declarations: a variable should be declared (in the same scope) at most once.
- undeclared variable: a variable should not be used before being declared.
- type mismatch: type of the left-hand side of an assignment should match the type of the right-hand side.
- wrong arguments: methods should be called with the right number and types of arguments.

The Classes of Errors (4)

3b. Semantic Errors

- Detected at Compile Time, and also add checks in code for runtime.

  Examples:

  - Division by 0
  - Array Index out-of-bound
  - Dereference of NULL pointer

- Can include code to check for conditions and not allow them to occur

  Advantage:

  - error message instead of error behavior

  Disadvantage:

  - object code is longer and slower

The Classes of Errors (5)

4. Logical Errors

- Hardest to detect
- Program is syntactically and semantically correct but does not do the "correct" thing
- Compiler can sometimes detect this problem.

Examples:

```
if (x = 0) result = 1;
result = 2;
```

* Assignment in the body of the if is "useless" – immediately overwritten by next assignment

2. SEMANTIC ANALYSIS

What is Semantic Analysis?

- Semantic analysis is the task of ensuring that the declarations and statements of a program are ***semantically*** correct, i.e, that their meaning is clear and consistent with the way in which control structures and data types are supposed to be used.

What Does Semantic Analysis Involve?

Semantic Analysis typically involves:

1. Type Checking:

- Data types are used in a manner that is consistent with their definition (i. e., only with compatible data types, only with operations that are defined for them, etc.)
- Error if operands are incompatible with the used operator.
- Example: 1.2 + 2 (real + int).

2. Label Checking:

- Labels references in a program must exist.

3. Flow control checks:

- control structures must be used in their proper fashion.
- E.g. No GOTOs into a FORTRAN DO statement, No breaks outside a loop or switch statement, break needs an enclosing loop, goto label needs a defined label. etc.)

Semantic Analysis – Type Checking

Semantic Analysis typically involves:

1. Type Checking:
   - Type Systems
   - Type Equivalence
   - Typing Expressions
     - Coercion
     - Overloading
     - Error Recovery
   - Typing Statements
   - Polymorphic Types

Type Checking (2)

Type Systems: System rules (Rules of a Language)

- E.g: Check "If both operands of the arithmetic operators of addition, subtraction and multiplication are of type integer, then the result is of type integer
- Definition:
  - What are the base/immutable types?
  - What type of constructors are available?
  - Can types be named? Or renamed?
- Resolutions:
  - What operators can be applied to what type?
  - What forms of coercion are allowed?
  - How are overloading situations resolved?

Type Checking (8)

- **Error Recovery**
  - As with parsing, often want to find multiple errors
  - What to do when one error detected?
    - Generally, return error as type but don't generate further errors
    - E.g., left argument of + returns error, still want the right argument to be some type that can be added.
  - Error recovery Modes:
    - Panic Mode: Skip symbols until input can by synchronized to a token
    - Phrase-level Recovery: Local error corrections, e.g. replacement of “**,**“by a “**;**“
    - Error Productions: Extension of grammar to handle common errors.
    - Global correction: Minimal correction of program in order to find a matching derivation (cost intensive)

Type Checking Situations

Some situations under which type checking is carried out are:

- Typing Expressions
- Operand Matching
- Operand Selection
- Coercion Types
- Selecting among Overload Possibilities
- Polymorphic Types Expansion

Forms Type Checking

- Static Type Checking - type checking done at compile type
  - Used in many strongly typed languages (where all variables/objects must have types).
- Dynamic Type Checking – done at runtime
  - Often used in languages where objects not strongly type (e.g., Lisp).
  - Type checking must be done at runtime (since objects not guaranteed to have a single type)

When To Type Check

- Depending on language
  - but some type checks can often be done in parsing
- Can also be done as a separate process
  - If done as a separate process, performed as a traversal of the Abstract Syntax Trees (AST) from the program
- Generally two routines:
  - Type of expressions
  - Type of statements

2.1 The Semantic Analyzer

- The Semantic Analyzer traverses the AST generated by the Syntax Analyzer (Parser), two (2) phases:

1. For each scope in the program:

- process the declarations =
  - add new entries to the symbol table and
  - report any variables that are multiply declared
- process the statements =
  - find uses of undeclared variables, and
  - update the "ID" nodes of the AST to point to the appropriate symbol- table entry.

2. Process all of the statements in program again:

- use the symbol-table information to determine the type of each expression, and to find type errors.

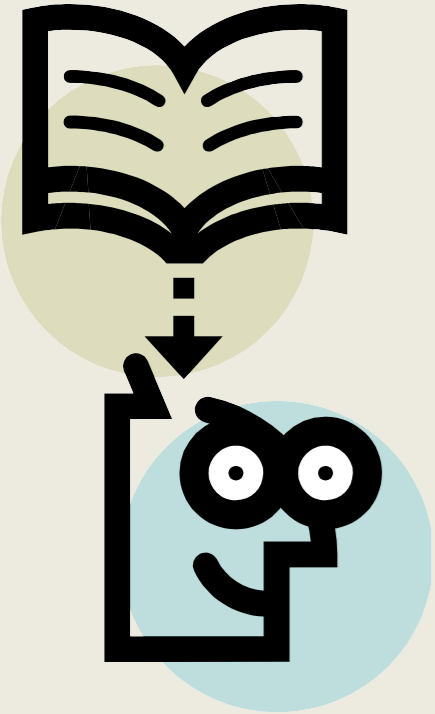
The Symbol Table

- The Symbol Table is the Set of Entries:

1. Keeps track of names declared in the program
2. Stores names of:
   - variables, classes, fields, methods, etc

3. Symbol table entry:
   - associates a name with a set of attributes, e.g.:
     - kind of name (variable, class, field, method, etc)
     - type (int, float, etc)
     - nesting level
     - memory location (i.e., where will it be found at runtime).

The Symbol Table



Please Read Up More On SYMBOL TABLE on the Learning resources on elearn platform

What Does Semantic Analysis Produce?

Part of semantic analysis is producing some sort of representation of the program –

* either object code or an intermediate representation of the program.

One-pass compilers will generate object code without using an intermediate representation;

* code generation is part of the semantic actions performed during parsing.

Other compilers (e.g. JAVA) will produce an intermediate representation during semantic analysis;

* most often it will be an AST or quadruples.

Semantic Analysis Example

Semantic Actions in Top-Down Parsing: An Example

Imagine we're parsing:

$S \rightarrow id := E$
$E \rightarrow T\ E'$
$E' \rightarrow + T\ E'$
$E' \rightarrow \varepsilon$
$T \rightarrow F\ T'$
$T' \rightarrow * F\ T'$
$T' \rightarrow \varepsilon$
$F \rightarrow id$
$F \rightarrow const$
$F \rightarrow (\ E\ )$

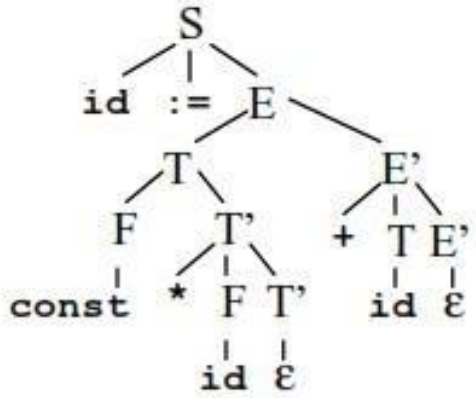
We insert the actions

$S \rightarrow id$ {*pushid*} := {*pushassn*} E {*buildassn*}
$E \rightarrow T\ E'$
$E' \rightarrow +$ {*pushop*} T {*buildexpr*} E'
$E' \rightarrow \varepsilon$
$T \rightarrow F\ T'$
$T' \rightarrow *$ {*pushop*} F {*buildterm*} T'
$T' \rightarrow \varepsilon$
$F \rightarrow id$ {*pushid*}
$F \rightarrow const$ {*pushconst*}
$F \rightarrow (\ E\ )$ {*pushfactor*}

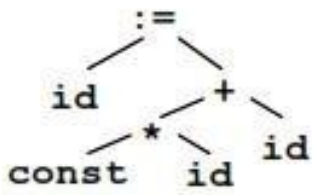
Semantic Analysis Example (2)

Example: Parsing An Expression

In parsing the expression `z := 2 * x + y`, we find this parse structure:



We want to create the AST fragment:



Semantic Analysis Example (3)

Parsing `z := 2*x + y` (continued)

Or we can produce a set of quadruples:

t_1 := 2 * x
t_2 := t_1 + y
z := t_2

Or we an produce a set of assembly language instructions:

```
mov  ax, 2
mov  bx, y
imul bx
mov  z, ax
```

3.0 Building the AST Example

1

Building the AST

z := 2 * x + y

S → id {*pushid*} := {*pushassn*} E {*buildassn*}

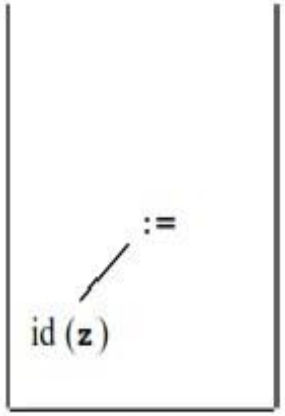
Semantic Stack

2

Building the AST (continued)

z := 2 * x + y

S → id {*pushid*} := {*pushassn*} E {*buildassn*}



Semantic Stack

Building the AST Example

3

Building the AST (continued)

`z := 2 * x + y`

S → id {*pushid*} := {*pushassn*} E {*buildassn*}

E → T E'

T → FT'

F → const {*pushconst*}

const (2)

:=

id (z)

Semantic Stack

4

Building the AST (continued)

`z := 2 * x + y`

S → id {*pushid*} := {*pushassn*} E {*buildassn*}

E → T E'

T → FT'

T' → * {*pushop*} F {*buildterm*} T'

*

const (2)

:=

id (z)

Semantic Stack

Building the AST Example

5

Building the AST (continued)

z := 2 * x + y

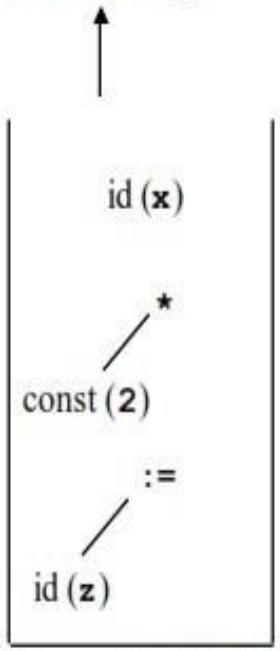
S → id {*pushid*} := {*pushassn*} E {*buildassn*}

E → T E'

T → FT'

T' → * {*pushop*} F {*buildterm*} T'

F → id {*pushid*}



Semantic Stack

6

Building the AST (continued)

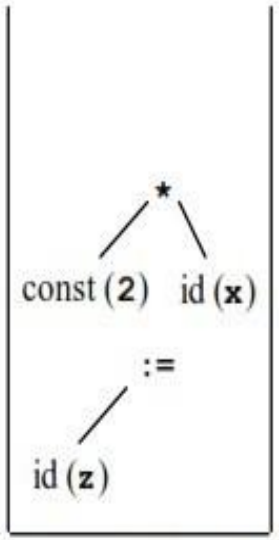
z := 2 * x + y

S → id {*pushid*} := {*pushassn*} E {*buildassn*}

E → T E'

T → FT'

T' → * {*pushop*} F {*buildterm*} T'



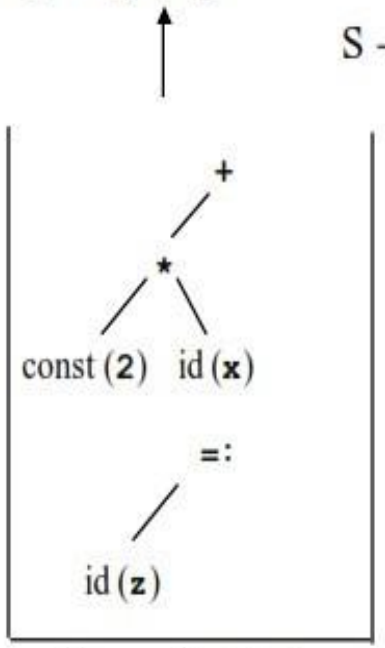
Semantic Stack

Building the AST Example

7

Building the AST (continued)

z := 2 * x + y



Semantic Stack

S → id {*pushid*} := {*pushassn*} E {*buildassn*}

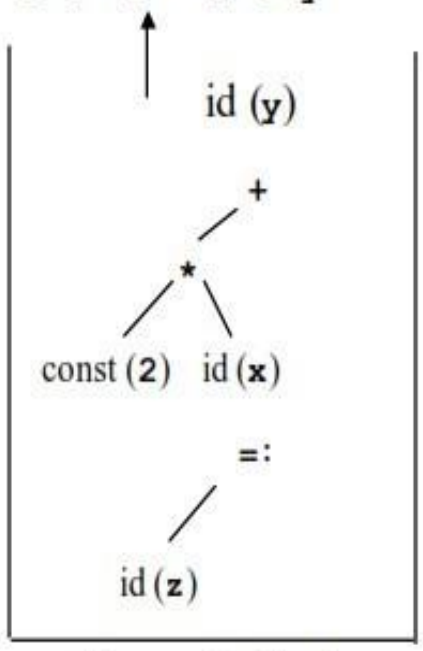
E → T E'

E' → + {*pushop*} T {*buildexpr*} E'

8

Building the AST (continued)

z := 2 * x + y



Semantic Stack

S → id {*pushid*} := {*pushassn*} E {*buildassn*}

E → T E'

E' → + {*pushop*} T {*buildexpr*} E'

T → FT'

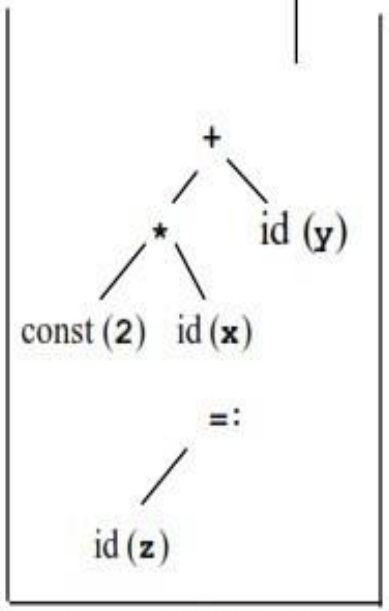
F → id {*pushid*}

Building the AST Example

9

Building the AST (continued)

z := 2 * x + y



Semantic Stack

S → id {*pushid*} := {*pushassn*} E {*buildassn*}

E → T E'

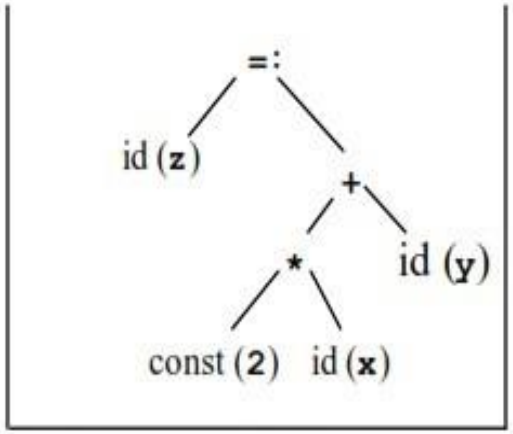
E' → + {*pushop*} T {*buildexpr*} E'

10

Building the AST (continued)

z := 2 * x + y

S → id {*pushid*} := {*pushassn*} E {*buildassn*}



Semantic Stack

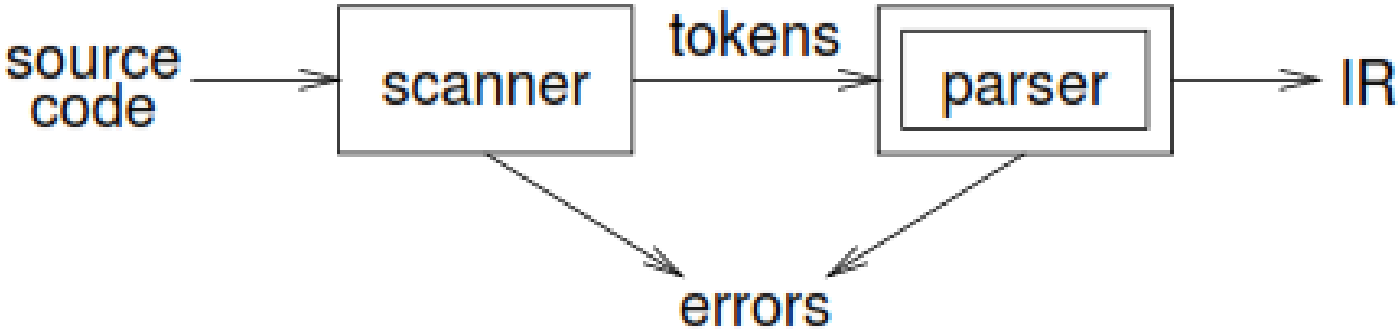
Decorating the AST

Abstract syntax trees have one enormous advantage over other intermediate representations:

* they can be “decorated”, i.e., each node on the AST can have their attributes saved in the AST nodes, which can simplify the task of type checking as the parsing process continues.

A Rejoinder

The role of the parser



Parser

- performs context-free syntax analysis
- guides context-sensitive analysis
- constructs an intermediate representation
- produces meaningful error messages
- attempts error correction